

PHASE 5

FitBuddy- AI Fitness Plan Generator using Gemini Models

Project Development

1. Introduction

The **Project Development Phase** is the stage where the requirements, design, and project plan created in the previous phases are converted into an actual working application.

For the **FitBuddy – AI Fitness Plan Generator**, this phase focuses on implementing the major components of the application, including the Generative AI functionality, FastAPI backend, database, routes, frontend templates, feedback mechanism, and dynamic Jinja2 rendering.

The documented FitBuddy implementation follows a modular architecture consisting of:

- Frontend using HTML and Jinja2.
- Backend using FastAPI.
- Generative AI using Google Gemini.
- Database using SQLite and SQLAlchemy.
- Local application serving through Uvicorn.

The development process is organized around the milestones established during the Project Planning Phase.

2. Purpose of Project Development

The main purpose of this phase is to implement the designed FitBuddy system.

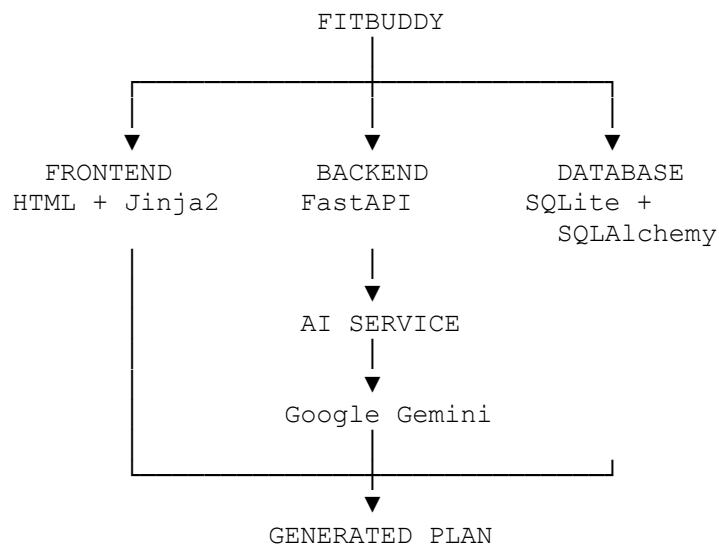
The development activities include:

- Setting up the project environment.
- Implementing AI-powered workout generation.
- Implementing nutrition/recovery-tip generation.
- Implementing feedback-based workout-plan updating.
- Creating database operations.
- Implementing FastAPI routes.
- Processing user input.
- Connecting AI functions with backend routes.
- Creating frontend pages.
- Implementing Jinja2 dynamic rendering.
- Connecting the frontend with the backend.
- Preparing the application for local execution.

The final objective is to integrate these components into a functional FitBuddy application.

3. Development Architecture

The development follows the architecture defined during the design phase.



The source describes the architecture as a modular FastAPI-based application with frontend, backend, AI, and database layers.

4. Development Environment Setup

Before implementing the application, the development environment is prepared.

The documented setup uses Python, pip, a virtual environment, and the required project libraries.

A virtual environment can be created using:

```
python -m venv fitbuddy-env
```

On Windows, the environment can be activated using:

```
fitbuddy-env\Scripts\activate
```

The documented dependencies include:

```
fastapi
uvicorn
jinja2
sqlalchemy
python-multipart
google-generativeai
```

These dependencies support the FastAPI backend, web templates, database, form processing, and Gemini integration.

5. Project Structure Development

The source documentation organizes the implementation around application files, templates, and static resources.

The documented structure includes:

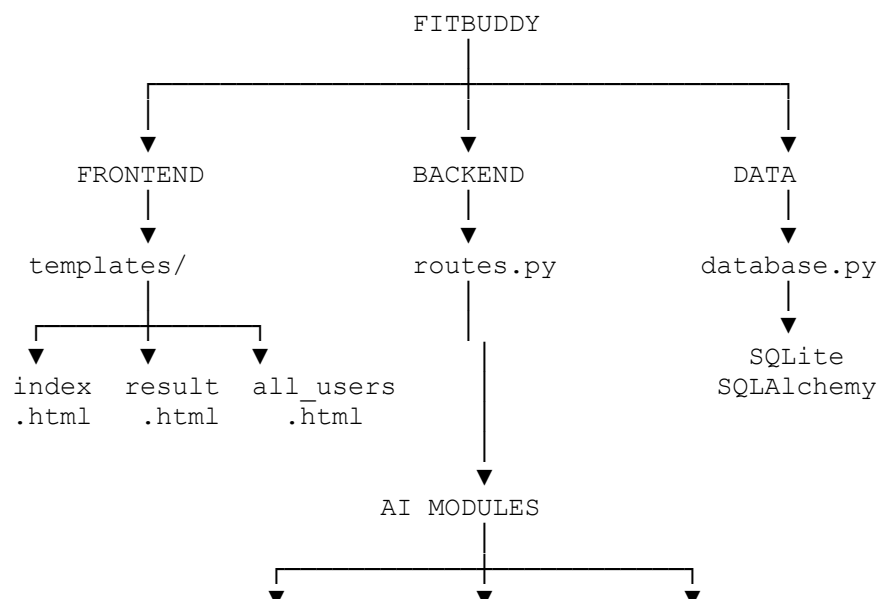
```
FitBuddy/
├── app/
│   └── Python application files
├── templates/
│   ├── index.html
│   ├── result.html
│   └── all_users.html
└── static/
    └── images/
```

The source also identifies specific implementation modules such as:

```
gemini_generator.py
gemini_flash_generator.py
updated_plan.py
database.py
routes.py
```

These modules separate AI generation, plan updating, database operations, and routing responsibilities.

6. Figure 1: FitBuddy Development Structure



7. Milestone 1 – Model and Architecture Implementation

The first development milestone establishes the technical foundation.

The source project identifies three activities:

1. Research and select the appropriate Generative AI model.
2. Define the application architecture.
3. Set up the development environment.

The selected Gemini models are documented as being used for different application functions.

8. Generative AI Model Implementation

The documented project experimented with multiple models before selecting Google Gemini.

The source identifies:

- T5
- BART
- DistilBERT
- Bloom

as models considered during experimentation.

The documented production selection was:

- **Gemini 1.5 Pro** for structured workout generation and plan updating.
- **Gemini Flash** for fast nutrition/recovery tips.

This division of AI functionality is implemented through separate Python functions.

9. Gemini API Configuration

The AI layer requires a Google Gemini API key.

The source documents the use of an environment variable:

```
GOOGLE_API_KEY=your_gemini_api_key_here
```

The key is intended to be supplied through environment configuration rather than being directly embedded in application logic.

This configuration allows the application to communicate with Google's Generative AI service.

10. Milestone 2 – Core Functionalities Development

The second milestone implements the major features of FitBuddy.

The core functionality includes:

1. Workout plan generation.
2. Nutrition/recovery-tip generation.
3. Feedback-based plan updating.
4. User storage.
5. Workout-plan storage.
6. FastAPI request processing.

These functions form the central working functionality of the application.

11. Workout Plan Generation

The main workout-generation function documented in the source is:

```
generate_workout_gemini()
```

The source associates this function with:

```
gemini_generator.py
```

Its purpose is to generate a personalized **7-day workout plan**.

The generation process uses information such as:

- Fitness goal.
- Workout intensity.
- User information.

The source identifies examples of fitness goals such as:

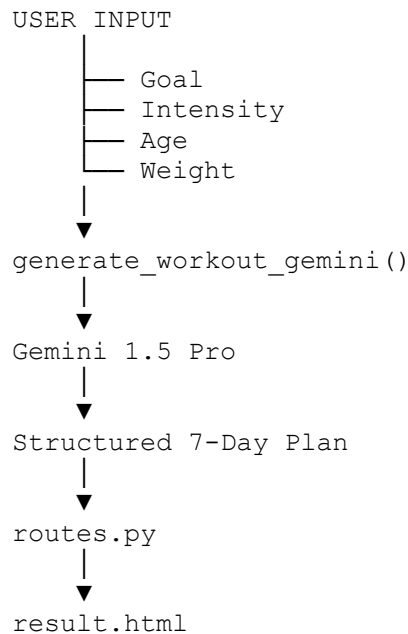
- Muscle gain.
- Weight loss.

The intensity can be:

- Low.
- Medium.
- High.

12. Workout Generation Processing

The workout-generation process is implemented as follows:



The generated response is returned to the routing layer and rendered on the result page.

13. Structure of the Generated Workout

The documented workout output includes day-wise information.

The source describes the generated plan as containing:

- Warm-up.
- Main workout.
- Exercise details.
- Sets.
- Repetitions.
- Cooldown or recovery guidance.

The workout is presented as a structured 7-day plan.

The source also describes examples of workout focus such as:

- Full body.
- Upper body.
- Lower body.
- Cardio.
- Core strength.
- Flexibility.

14. Nutrition Tip Development

A separate function is implemented for nutrition/recovery advice:

```
generate_nutrition_tip_with_flash()
```

The source associates this function with:

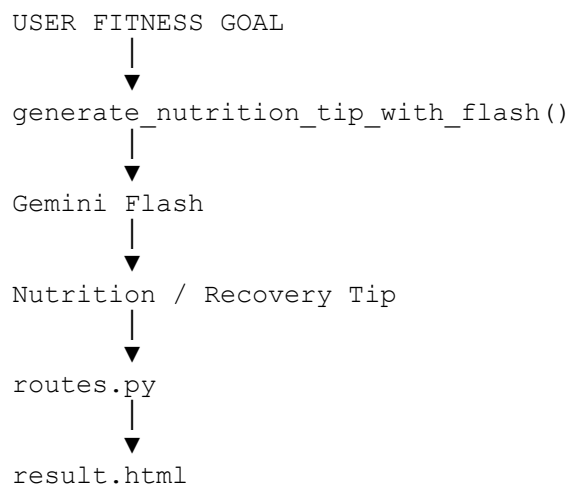
```
gemini_flash_generator.py
```

Its purpose is to generate a concise and practical nutrition or recovery tip based on the user's selected fitness goal.

Examples described by the source include:

- Hydration advice.
- Protein suggestions.
- General recovery practices.

15. Nutrition Generation Flow



The nutrition tip is displayed below the workout plan on the result page.

16. Feedback-Based Workout Development

FitBuddy includes an important feedback mechanism.

The documented function is:

```
update_workout_plan()
```

The source associates this function with:

```
updated_plan.py
```

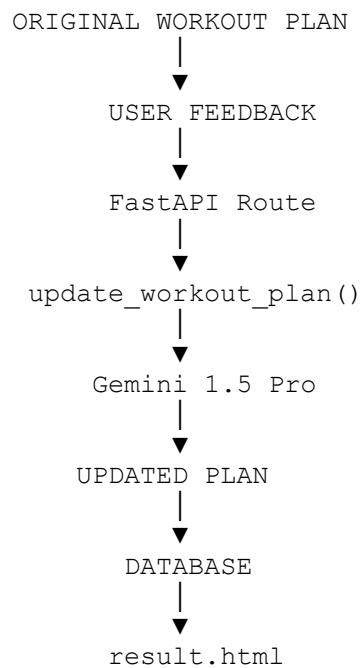
This function allows the user to request modifications to an existing workout plan.

Examples of feedback described in the source include:

- "Add yoga"
- "Include more cardio"
- More focus on cardio.
- More rest days.

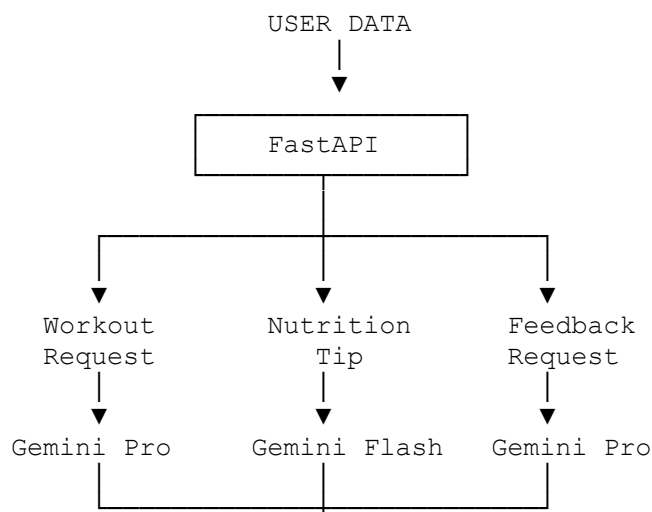
17. Feedback Processing Flow

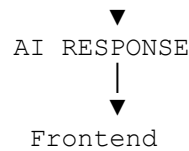
The feedback implementation follows this sequence:



The source states that the original plan and feedback are passed to Gemini 1.5 Pro to produce a modified plan.

18. Figure 2: AI Development Flow





19. User and Plan Storage Development

The application uses SQLite and SQLAlchemy for persistent storage.

The documented database functions include:

```
save_user()
save_plan()
update_plan()
get_original_plan()
get_user()
```

These operations allow the application to store and retrieve user and workout information.

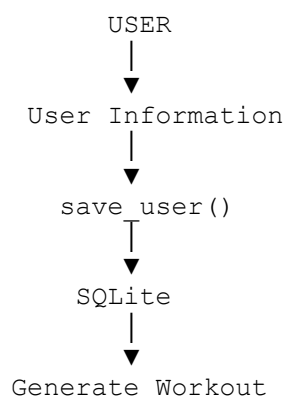
20. Database Development

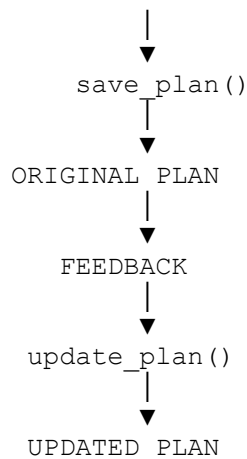
The database stores information including:

- User ID.
- Name.
- Age.
- Weight.
- Fitness goal.
- Workout intensity.
- Original workout plan.
- Updated workout plan.

The source states that feedback-based updates are stored separately so that both the original and updated versions can be preserved.

21. Figure 3: Database Development Flow





22. FastAPI Backend Development

FastAPI is implemented as the main backend framework.

The backend is responsible for:

- Receiving form data.
- Processing user requests.
- Calling Gemini functions.
- Storing information.
- Retrieving information.
- Rendering templates.

The source states that routing logic is handled in `routes.py`.

23. User Input Processing

The frontend form collects:

- Username.
- User ID.
- Age.
- Weight.
- Fitness goal.
- Workout intensity.

FastAPI captures these values using `Form(...)` parameters.

The source also documents Pydantic structures including:

```
UserInput  
FeedbackRequest
```

These are used to represent and validate the structure of the application inputs.

24. Backend Route Development

The documented routes are:

Route	Development Purpose
/	Display the user form
/generate-workout	Generate workout and nutrition output
/submit-feedback	Update workout using feedback
/view-all-users	Display users and their plans

These routes form the main operational interface between the frontend and backend.

25. Home Route Development

The / route is responsible for displaying the main input page.

The page is:

`index.html`

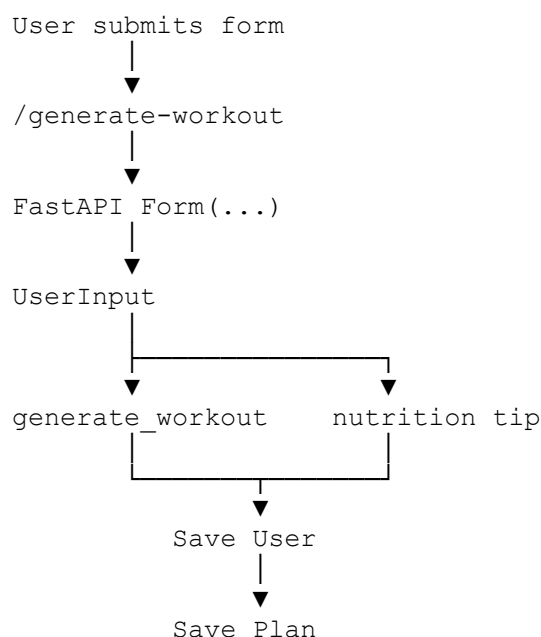
The page provides the user with a form for entering fitness-related information.

No workout generation takes place until the user submits the form.

26. Workout Generation Route

The /generate-workout route processes the submitted user information.

The documented process is:



↓
result.html

The source explicitly describes the route as calling both workout-generation and nutrition-tip functions and then storing the user and plan.

27. Feedback Route Development

The `/submit-feedback` route is responsible for processing plan modification requests.

The documented process is:

1. Capture feedback and user ID.
2. Retrieve the original plan.
3. Send the original plan and feedback to `update_workout_plan()`.
4. Receive the revised plan.
5. Update the database.
6. Render the updated result.

This connects the feedback interface with the AI and database layers.

28. Admin View Development

The `/view-all-users` route provides access to user and plan information.

The source documents the following database functions for this functionality:

```
get_all_users()  
get_all_plans()
```

The results are passed to:

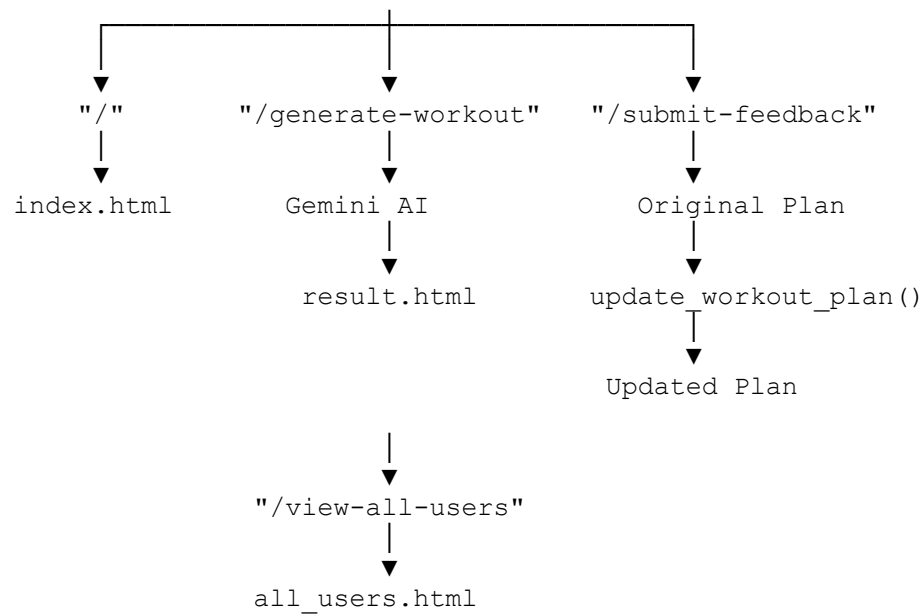
```
all_users.html
```

The page allows administrators or developers to view information including:

- Username.
- Age.
- Weight.
- Goal.
- Intensity.
- Original workout plan.
- Updated workout plan.

29. Figure 4: FastAPI Route Integration

FastAPI
|



30. Frontend Development

The frontend is developed using:

- HTML.
- CSS.
- Jinja2.

The source describes three principal templates:

```

index.html
result.html
all_users.html

```

These templates support the main user workflow and administrative view.

31. `index.html` Development

The `index.html` page is developed as the main user input interface.

The documented form collects:

- Name.
- User ID.
- Age.
- Weight.
- Fitness goal.
- Workout intensity.

The form submits the information to:

```
/generate-workout
```

This allows the backend to process the submitted information.

32. `result.html` Development

The `result.html` page displays the generated information.

The source describes the page as displaying:

- User information.
- 7-day workout plan.
- Nutrition tip.
- Feedback form.

The generated workout is displayed in a structured format, while the nutrition tip appears below the workout plan.

33. Dynamic Template Development

FastAPI's Jinja2 template system is used to connect backend information to HTML.

The source documents:

```
Jinja2Templates
```

and a template directory.

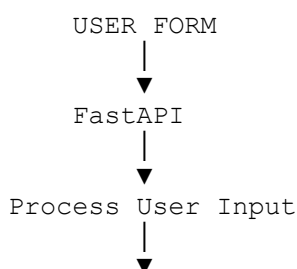
The backend passes information to the templates using context values.

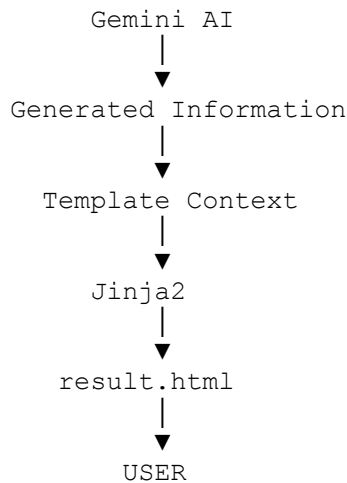
Examples documented in the source include:

```
{{ username }}
{{ goal }}
{{ intensity }}
{{ workout_plan }}
{{ nutrition_tip }}
```

This allows the generated information to appear dynamically on the result page.

34. Figure 5: Jinja2 Development Flow





35. Frontend Styling Development

The source describes a fitness-oriented interface with:

- Dark background.
- Roboto font.
- Flexbox.
- Responsive media queries.
- Rounded corners.
- Shadows.
- Hover effects.

The purpose of the styling is to create a clean and engaging fitness-focused interface.

36. Personalized Workout Display

Once the user submits the form, the application displays a personalized workout page.

The source describes a user-information section showing:

- Name.
- User ID.
- Age.
- Weight.
- Fitness goal.
- Selected intensity.

This information provides context for the generated workout plan.

37. Workout Result Display

The workout result page presents a day-wise fitness schedule.

The documented output may contain:

- Workout focus.
- Exercise names.
- Sets.
- Repetitions or duration.
- Rest intervals.
- Warm-up guidance.
- Cool-down guidance.

The source describes the purpose as providing an easy-to-follow, goal-oriented weekly schedule.

38. Nutrition Result Display

The nutrition section provides concise dietary or recovery information.

The source describes the nutrition guidance as complementary to the workout plan and gives examples such as prioritizing protein sources including:

- Chicken.
- Fish.
- Beans.
- Greek yogurt.

The source presents this as guidance supporting fitness goals such as muscle building and fat loss.

39. Feedback Interface Development

The feedback interface allows users to refine their workout plans.

The user provides:

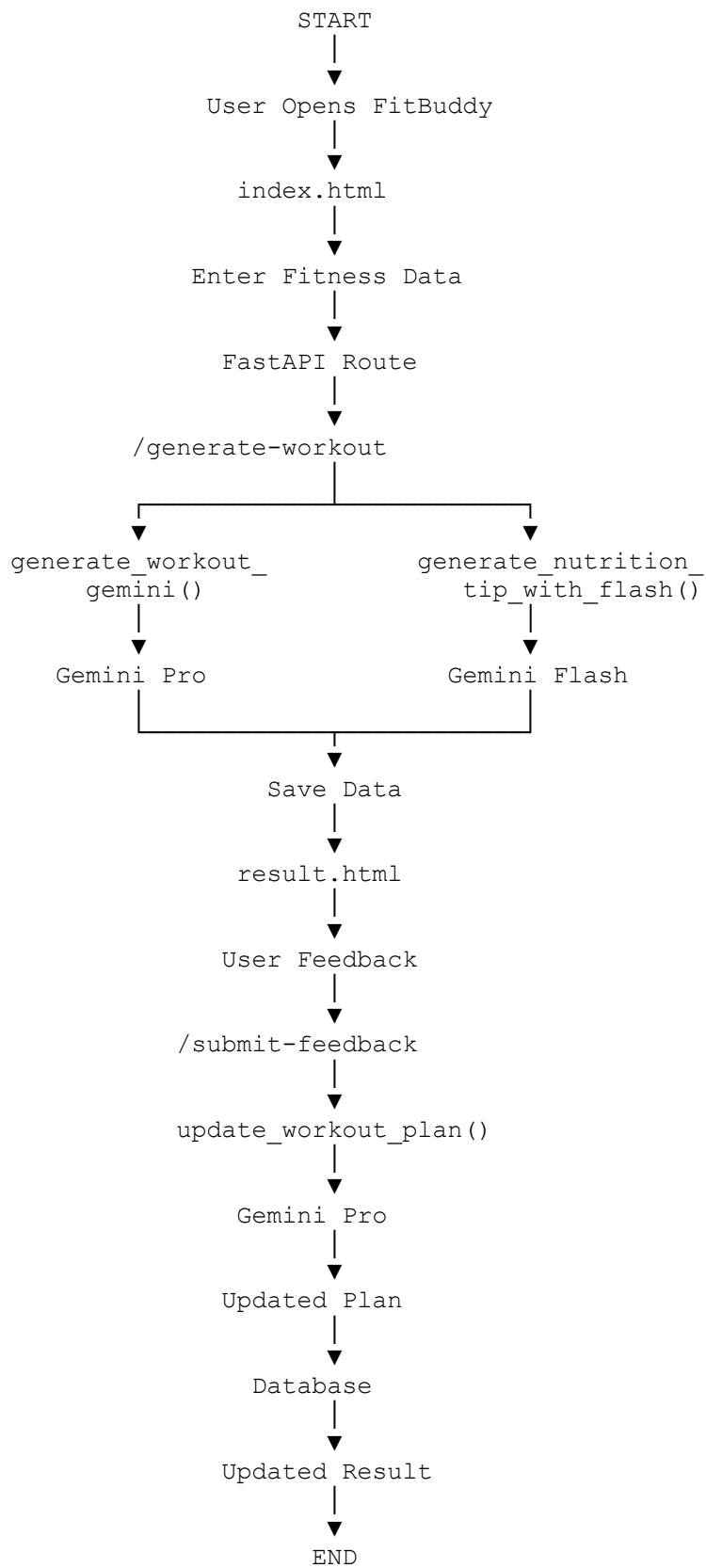
- User ID.
- Feedback about the current workout plan.

The feedback may concern:

- Effectiveness.
- Difficulty.
- Preferences.
- Desired changes.

The information is then processed by the backend and used to update the workout plan through Generative AI.

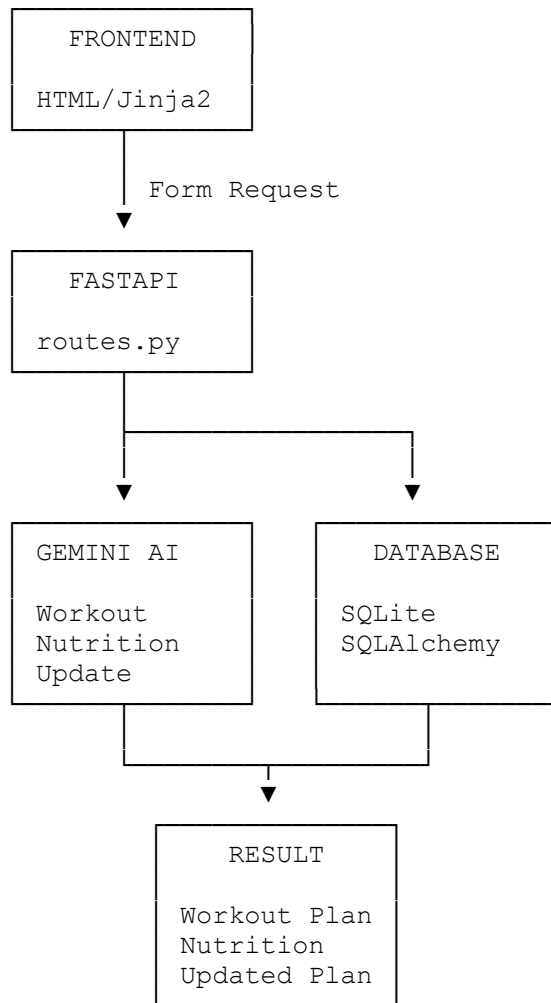
40. Figure 6: Complete Development Workflow



41. Integration of Frontend, Backend, AI and Database

The development phase brings the individual components together.

The complete interaction is:



42. Development of the Admin View

The `all_users.html` page is developed to provide a consolidated view of stored user information.

The source describes the use of a Jinja2 loop:

```
{% for user in users %}
```

This allows multiple user records to be displayed dynamically.

The page provides visibility into original and updated workout plans

43. Local Application Development

After the major components are implemented, the application is prepared for local execution.

The documented development server uses Uvicorn.

The source gives the local application address as:

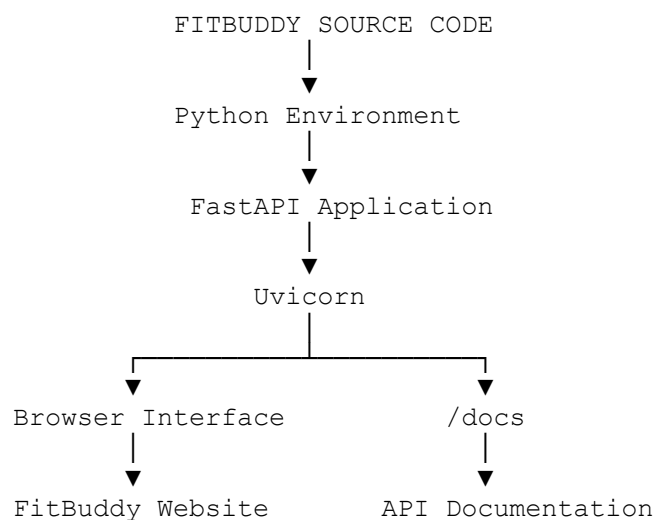
```
http://127.0.0.1:8000
```

FastAPI's interactive documentation is available at:

```
http://127.0.0.1:8000/docs
```

These provide access to the application interface and API documentation respectively.

44. Figure 7: Local Development Environment



45. Development of the Complete User Journey

The development implementation connects the individual components into a complete user experience.

Step 1

The user opens the FitBuddy homepage.

Step 2

The user enters personal and fitness information.

Step 3

The form submits the information to the FastAPI backend.

Step 4

The backend creates the required input structure.

Step 5

The workout-generation function sends the relevant request to Gemini.

Step 6

The nutrition function generates a complementary tip.

Step 7

The user and generated plan are stored in SQLite.

Step 8

The result page displays the generated information.

Step 9

The user can submit feedback.

Step 10

The original plan and feedback are passed to the plan-update function.

Step 11

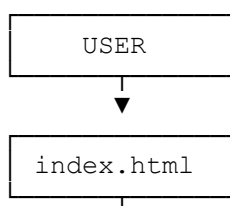
Gemini generates the revised plan.

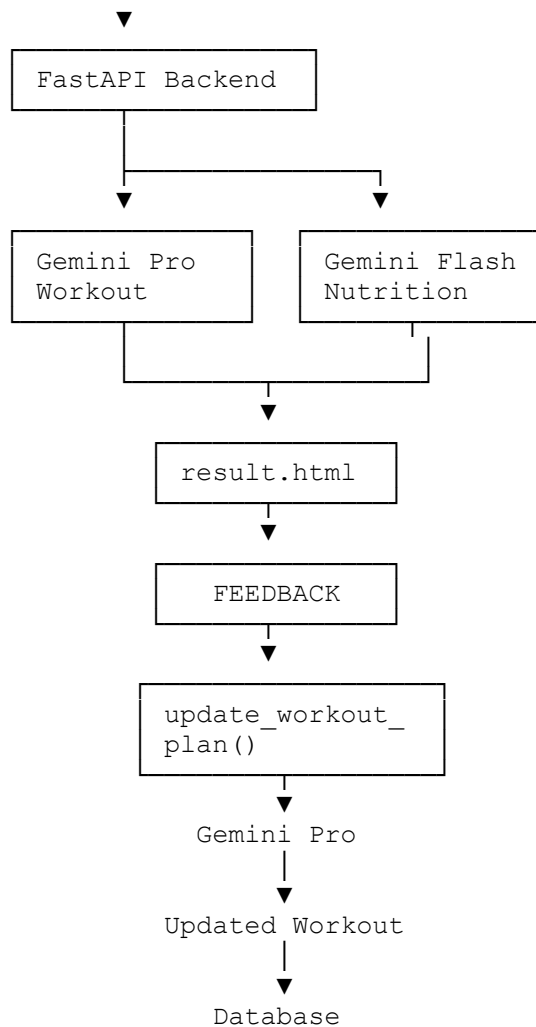
Step 12

The updated plan is stored and displayed.

This workflow reflects the implementation described in the source document.

46. Figure 8: End-to-End Development Flow





47. Development Integration Testing Preparation

Although detailed test results belong to the Project Testing Phase, basic verification is required during development to confirm that components communicate correctly.

The development process therefore prepares the following areas for testing:

- Form submission.
- FastAPI route processing.
- Gemini API calls.
- Workout generation.
- Nutrition-tip generation.
- Database storage.
- Feedback processing.
- Updated-plan storage.
- Jinja2 rendering.
- Local application access.

The source specifically identifies local deployment testing and verification as a later milestone.

48. Development Challenges and Considerations

During implementation, several technical areas require attention.

48.1 AI Integration

The Gemini API must receive appropriate information and return usable generated content.

48.2 Structured Output

The generated workout must be presented in a readable day-wise structure.

48.3 Database Persistence

The original and updated plans need to be stored appropriately.

48.4 Backend-Frontend Communication

The FastAPI routes must correctly pass generated information to Jinja2 templates.

48.5 Feedback Processing

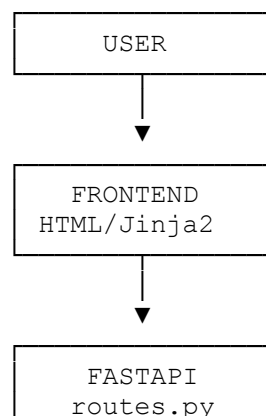
The feedback mechanism must identify the relevant original plan before requesting an update.

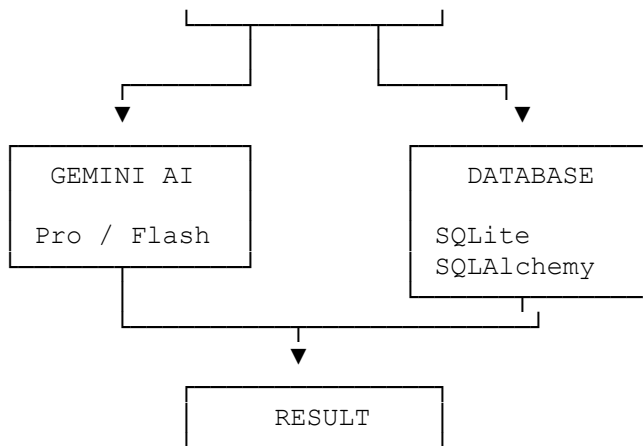
48.6 Environment Configuration

The Gemini API key and required dependencies must be correctly configured.

These are development considerations derived from the implementation structure documented in the source.

49. Figure 9: Component Integration





50. Development Phase Deliverables

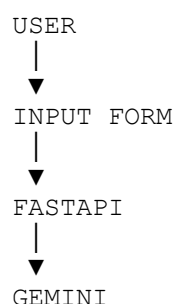
At the end of the Project Development Phase, the planned implementation produces the following major components:

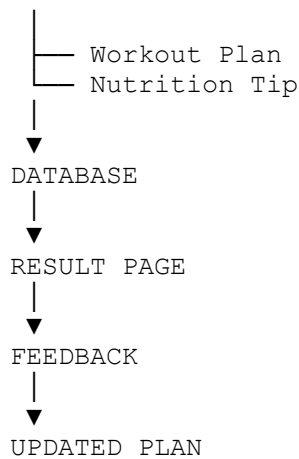
1. FastAPI backend.
2. AI-powered workout-generation functionality.
3. Nutrition/recovery-tip functionality.
4. Feedback-based workout-plan updating.
5. SQLite database.
6. SQLAlchemy database operations.
7. FastAPI route handlers.
8. User input processing.
9. Jinja2 dynamic templates.
10. User-facing workout result page.
11. Feedback interface.
12. All-users administrative view.
13. Local Uvicorn execution setup.
14. Integrated frontend-backend-AI-database workflow.

These components represent the implementation of the design and development plan.

51. Development Phase Output

The development process produces a working application structure in which:





This represents the core operational cycle of FitBuddy.

52. Transition to Phase 6 – Project Testing

After the development phase, the next stage is **Phase 6 – Project Testing**.

The testing phase will verify whether the implemented FitBuddy components work as intended.

Testing will focus on:

- User input.
- Route functionality.
- AI generation.
- Nutrition-tip generation.
- Feedback-based plan updating.
- Database operations.
- Jinja2 rendering.
- Frontend functionality.
- Local deployment.
- API endpoints.

The testing phase will identify errors or unexpected behavior and verify that the developed application meets the requirements established during Phase 2.

53. Phase 5 Conclusion

The **Project Development Phase** transforms the FitBuddy requirements, design, and development plan into an implemented application.

The development integrates the major technologies documented in the source project: **FastAPI for backend processing, Google Gemini for Generative AI, SQLite and SQLAlchemy for persistence, and HTML/Jinja2 for the frontend.**

The main implemented functionality includes personalized 7-day workout generation, nutrition/recovery-tip generation, feedback-based workout-plan updating, database persistence, FastAPI routing, and dynamic frontend rendering.